

STRUCTIFY

Backend Engineering Document

Testing Strategy & Infrastructure Guide

v1.0.0 · Juni 2026 · Lionel Jevoon

Laravel 10 · PHP 8.2 · MySQL · Sanctum

Type Dokumen	Software Engineering Document — Backend
Project	Structify — Platform Belajar Struktur Data & Algoritma
Developer	Lionel Jevoon
Stack	Laravel 10, PHP 8.2, MySQL, Laravel Sanctum
Status	In Development — Draft v1.0.0
Tanggal	Juni 2026

Dokumen ini merupakan panduan teknis engineer untuk backend Structify, mencakup arsitektur infrastruktur, strategi testing, standar API, dan prosedur deployment. Dokumen bersifat living document dan akan diperbarui seiring pengembangan.

Daftar Isi

01 — Arsitektur & Infrastruktur Backend

- 1.1 — System Architecture Overview
- 1.2 — Tech Stack Detail
- 1.3 — Directory Structure
- 1.4 — Environment Configuration

02 — Database Infrastructure

- 2.1 — Schema Overview
- 2.2 — Migration Strategy
- 2.3 — Indexing & Performance
- 2.4 — Seeding Strategy

03 — API Infrastructure

- 3.1 — Route Architecture
- 3.2 — Authentication Flow (Sanctum)
- 3.3 — Middleware Stack
- 3.4 — Request & Response Standards
- 3.5 — Error Handling

04 — Testing Strategy

- 4.1 — Testing Pyramid
- 4.2 — Unit Testing
- 4.3 — Feature Testing
- 4.4 — API Endpoint Testing
- 4.5 — Test Coverage Targets

05 — Security Infrastructure

- 5.1 — Authentication & Authorization
- 5.2 — Input Validation
- 5.3 — Rate Limiting
- 5.4 — CORS Configuration

06 — Deployment & DevOps

- 6.1 — Environment Strategy
- 6.2 — Deployment Checklist
- 6.3 — Monitoring & Logging

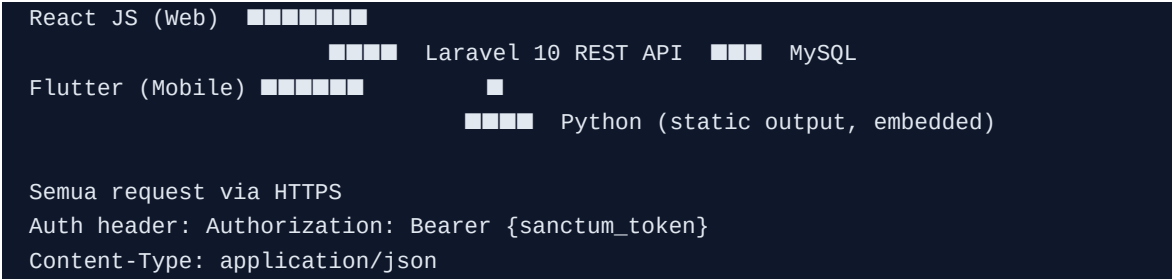
01 Arsitektur & Infrastruktur Backend

1.1 — System Architecture Overview

Structify menggunakan arsitektur monolith RESTful API berbasis Laravel 10. Backend bertindak sebagai single source of truth untuk seluruh data dan business logic, dikonsumsi oleh dua klien: React JS (web) dan Flutter (mobile). Python tidak di-deploy sebagai service — hanya digunakan untuk generate static output yang kemudian di-embed langsung ke konten cheatsheet.

Layer	Teknologi	Peran	Catatan
Client Web	React JS + Vite	UI/UX — consume REST API	Tailwind CSS
Client Mobile	Flutter (Dart)	Android & iOS — consume REST API	Material 3
Backend API	Laravel 10	REST API, Auth, Business Logic	PHP 8.2
Auth	Laravel Sanctum	Token-based auth (SPA & Mobile)	Bearer Token
Database	MySQL 8.0	Persistent storage	Via Eloquent ORM
Logic Engine	Python	Algoritma & struktur data	Static output only

1.2 — Communication Flow



1.3 — Directory Structure

```
structify-backend/
├── app/
│   ├── Http/
│   │   ├── Controllers/      # AuthController, TrackController, ...
│   │   ├── Middleware/      # IsAdmin, Authenticate
│   │   └── Requests/        # RegisterRequest, LoginRequest, ...
│   └── Models/              # User, Track, Topic, Question, ...
├── database/
│   ├── migrations/          # Semua tabel migration
│   └── seeders/             # TrackSeeder, TopicSeeder, AdminSeeder
├── routes/
├── api.php                  # Semua API routes
└── tests/
```

```
■ ■■■ Feature/           # Endpoint & integration tests
■ ■■■ Unit/             # Unit tests untuk logic
■■■ .env                 # Environment variables
```

1.4 — Environment Configuration

File `.env` harus dikonfigurasi sesuai environment. Jangan pernah commit `.env` ke repository. Gunakan `.env.example` sebagai template.

```
# App
APP_NAME=Structify
APP_ENV=local           # local | staging | production
APP_KEY=base64:...      # php artisan key:generate
APP_DEBUG=true          # false di production
APP_URL=http://localhost:8000

# Database
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=structify_db
DB_USERNAME=root
DB_PASSWORD=

# Sanctum
SANCTUM_STATEFUL_DOMAINS=localhost:5173,localhost:3000

# Session
SESSION_DRIVER=cookie
SESSION_LIFETIME=120
```

02 Database Infrastructure

2.1 — Schema Overview

Database Structify terdiri dari 9 tabel utama. Seluruh tabel menggunakan InnoDB engine dengan charset utf8mb4 untuk support emoji dan karakter Unicode penuh.

Tabel	Kolom Utama	Relasi
users	id, name, email, password, role	HasMany: quiz_attempts, forum_posts, user_progress
tracks	id, name, slug, description, icon	HasMany: topics
topics	id, track_id, title, slug, display_order	BelongsTo: track HasMany: questions
questions	id, topic_id, question_text, option_a/b/c/d, correct_answer, timer_seconds	BelongsTo: topic
quiz_attempts	id, user_id, topic_id, score, total_questions, completed_at	BelongsTo: user, topic
quiz_attempt_answers	id, quiz_attempt_id, question_id, selected_answer	BelongsToMany: quiz_attempt, question
forum_posts	id, user_id, topic_id, parent_id, title, body, upvotes	Self-referencing (replies)
tags	id, name, slug	BelongsToMany: forum_posts
forum_post_tags	forum_post_id, tag_id	Pivot table
user_progress	id, user_id, topic_id, is_completed, completion_timestamp, last_visited_at	BelongsToMany: user, topic

2.2 — Migration Strategy

Urutan migration sangat penting karena foreign key constraints. Tabel parent harus selalu di-migrate sebelum tabel child.

Urutan	File Migration	Keterangan
1	create_users_table	Tabel utama — tidak ada FK
2	create_tracks_table	Parent dari topics
3	create_topics_table	FK: track_id → tracks.id
4	create_questions_table	FK: topic_id → topics.id
5	create_quiz_attempts_table	FK: user_id, topic_id
6	create_quiz_attempt_answers_table	FK: quiz_attempt_id, question_id
7	create_tags_table	Tidak ada FK
8	create_forum_posts_table	FK: user_id, topic_id, parent_id (self)
9	create_forum_post_tags_table	Pivot — FK: forum_post_id, tag_id

10	create_user_progress_table	FK: user_id, topic_id
----	----------------------------	-----------------------

2.3 — Indexing & Performance

Index ditambahkan pada kolom yang sering digunakan di WHERE clause dan JOIN. Kolom slug sudah unique (otomatis indexed). Berikut index tambahan yang direkomendasikan:

```
-- Index untuk query yang sering digunakan
ALTER TABLE topics ADD INDEX idx_track_id (track_id);
ALTER TABLE topics ADD INDEX idx_display_order (display_order);
ALTER TABLE questions ADD INDEX idx_topic_id (topic_id);
ALTER TABLE quiz_attempts ADD INDEX idx_user_topic (user_id, topic_id);
ALTER TABLE forum_posts ADD INDEX idx_topic_id (topic_id);
ALTER TABLE forum_posts ADD INDEX idx_parent_id (parent_id);
ALTER TABLE user_progress ADD INDEX idx_user_topic (user_id, topic_id);
```

2.4 — Seeding Strategy

Seeder dibagi menjadi dua kategori: data permanen yang selalu dijalankan dan data testing yang hanya dijalankan di environment development.

Seeder	Data	Environment
TrackSeeder	4 track utama (Struktur Data, Algoritma, Diskrit, OOP)	All
TopicSeeder	41 topik tersebar di 4 track	All
AdminSeeder	1 akun admin default	All
QuestionSeeder	Sample soal kuis per topik	Dev only
UserSeeder	Sample user untuk testing	Dev only

03 API Infrastructure

3.1 — Route Architecture

Semua route API berada di `routes/api.php` dengan prefix `/api` secara otomatis. Route dikelompokkan berdasarkan level akses: public, authenticated user, dan admin.

Public Routes (tidak perlu token):

```
POST /api/register
POST /api/login
GET /api/tracks
GET /api/tracks/{slug}
GET /api/tracks/{slug}/topics
GET /api/topics/{slug}
```

Authenticated Routes (Bearer token required):

```
POST /api/logout
GET /api/me
GET /api/me/progress
PATCH /api/topics/{id}/progress
POST /api/topics/{id}/visited
GET /api/topics/{id}/questions
POST /api/quiz/submit
GET /api/me/quiz-history
GET /api/me/quiz-history/{id}
GET /api/forum
POST /api/forum
GET /api/forum/{id}
POST /api/forum/{id}/reply
POST /api/forum/{id}/upvote
```

Admin Routes (role: admin | developer):

```
GET /api/admin/questions
POST /api/admin/questions
PUT /api/admin/questions/{id}
DELETE /api/admin/questions/{id}
GET /api/admin/users
PATCH /api/admin/users/{id}/ban
PATCH /api/admin/users/{id}/promote
GET /api/admin/analytics
```

3.2 — Authentication Flow (Sanctum)

Structify menggunakan Laravel Sanctum untuk token-based authentication. Token dibuat saat register/login dan harus dikirim di setiap request ke protected route melalui Authorization header.

1. Client kirim POST `/api/login` dengan email & password
2. Backend validasi kredensial
3. Backend generate Sanctum token

4. Client simpan token (React: localStorage | Flutter: secure_storage)
5. Setiap request selanjutnya sertakan header:
Authorization: Bearer {token}
6. POST /api/logout menghapus token dari database

3.3 — Middleware Stack

Middleware	Fungsi	Digunakan pada
auth:sanctum	Verifikasi Bearer token Sanctum	Semua protected route
IsAdmin	Cek role user adalah admin/developer	Semua /admin/* route
throttle:api	Rate limiting default Laravel	Semua API route

3.4 — Request & Response Standards

Standar format response JSON untuk semua endpoint:

```
// Success response
{ "message": "...", "data": {...} }           // 200 OK
{ "message": "...", "data": {...} }           // 201 Created

// Error response
{ "message": "Unauthenticated." }             // 401
{ "message": "Unauthorized. Admin only." }     // 403
{ "message": "No query results for ..." }    // 404
{ "errors": { "field": ["msg"] } }             // 422 Validation
{ "message": "Server Error" }                 // 500
```

3.5 — Error Handling

Laravel secara otomatis handle exception dan return JSON response untuk API request. Konfigurasi tambahan di app/Exceptions/Handler.php untuk standarisasi format error.

```
// app/Exceptions/Handler.php
public function render($request, Throwable $e) {
    if ($request->expectsJson()) {
        if ($e instanceof ModelNotFoundException) {
            return response()->json(['message' => 'Resource not found'], 404);
        }
        if ($e instanceof AuthorizationException) {
            return response()->json(['message' => 'Unauthorized'], 403);
        }
    }
    return parent::render($request, $e);
}
```


04 Testing Strategy

4.1 — Testing Pyramid

Structify menggunakan pendekatan testing pyramid dengan tiga layer. Fokus utama ada di Feature Tests karena ini adalah API-only backend — testing endpoint langsung memberikan confidence paling tinggi.



4.2 — Unit Testing

Unit test memastikan logika bisnis terisolasi berjalan benar tanpa dependencies eksternal seperti database atau HTTP.

Setup test environment:

```
# File: phpunit.xml
<php>
  <env name="APP_ENV" value="testing"/>
  <env name="DB_CONNECTION" value="sqlite"/>
  <env name="DB_DATABASE" value=":memory:"/>
</php>
```

Contoh Unit Test — Quiz Score Calculation:

```
// tests/Unit/QuizScoreTest.php
class QuizScoreTest extends TestCase
{
    public function test_score_calculated_correctly(): void
    {
        $correct = 17;
        $total   = 20;
        $score    = ($correct / $total) * 100;

        $this->assertEquals(85.00, $score);
    }

    public function test_perfect_score(): void
    {
        $score = (20 / 20) * 100;
        $this->assertEquals(100.00, $score);
    }

    public function test_zero_score(): void
    {

```

```

    {
        $score = (0 / 20) * 100;
        $this->assertEquals(0.00, $score);
    }
}

```

4.3 — Feature Testing

Feature test mensimulasikan HTTP request ke endpoint dan memverifikasi response. Menggunakan in-memory SQLite untuk kecepatan dan isolasi.

Contoh Feature Test — Auth:

```

// tests/Feature/AuthTest.php
class AuthTest extends TestCase
{
    use RefreshDatabase;

    public function test_user_can_register(): void
    {
        $response = $this->postJson('/api/register', [
            'name' => 'Lionel',
            'email' => 'lionel@example.com',
            'password' => 'password123',
            'password_confirmation' => 'password123',
        ]);

        $response->assertStatus(201)
            ->assertJsonStructure([
                'message', 'user' => ['id', 'name', 'email', 'role'], 'token'
            ]);
    }

    public function test_user_can_login(): void
    {
        $user = User::factory()->create(['password' => bcrypt('password123')]);

        $response = $this->postJson('/api/login', [
            'email' => $user->email,
            'password' => 'password123',
        ]);

        $response->assertStatus(200)->assertJsonStructure(['token']);
    }

    public function test_login_fails_with_wrong_password(): void
    {
        $user = User::factory()->create();

        $this->postJson('/api/login', [
            'email' => $user->email,
            'password' => 'wrongpassword',
        ]);
    }
}

```

```

    ]->assertStatus(422);
  }
}

```

4.4 — API Endpoint Testing (Checklist)

Setiap endpoint harus memiliki minimal 3 test case: happy path, unauthorized access, dan invalid input.

Endpoint	Test Case	Expected Status
POST /register	Data valid	201
POST /register	Email sudah terdaftar	422
POST /register	Password terlalu pendek	422
POST /login	Kredensial benar	200
POST /login	Password salah	422
GET /me	Token valid	200
GET /me	Tanpa token	401
GET /tracks	Public — tanpa token	200
GET /tracks/{slug}	Slug valid	200
GET /tracks/{slug}	Slug tidak ada	404
PATCH /topics/{id}/progress	Token valid, data benar	200
PATCH /topics/{id}/progress	Tanpa token	401
POST /admin/questions	Role admin	201
POST /admin/questions	Role user biasa	403
POST /quiz/submit	Jawaban lengkap	200
POST /quiz/submit	topic_id tidak ada	404

4.5 — Test Coverage Targets

Modul	Target Coverage	Prioritas
Auth (register, login, logout, me)	100%	Wajib
Track & Topic endpoints	90%+	Wajib
Quiz submit & scoring logic	100%	Wajib
Admin middleware & CRUD soal	90%+	Wajib
User Progress tracking	85%+	Tinggi

Forum posts & replies	80%+	Sedang
Admin analytics	70%+	Rendah

Cara jalankan test:

```
# Jalankan semua test
php artisan test

# Jalankan test spesifik
php artisan test --filter AuthTest

# Dengan coverage report (butuh Xdebug)
php artisan test --coverage

# Parallel testing (lebih cepat)
php artisan test --parallel
```

05 Security Infrastructure

5.1 — Authentication & Authorization

Aspek	Implementasi
Password hashing	bcrypt via Hash::make() — cost factor 12 (default Laravel)
Token storage	Sanctum tokens disimpan di personal_access_tokens table (hashed)
Token lifetime	Default tidak expired — bisa di-set via Sanctum config
Role check	Middleware IsAdmin cek field role di tabel users
Token revocation	currentAccessToken()->delete() saat logout

5.2 — Input Validation

Semua input user divalidasi menggunakan Laravel Form Request sebelum masuk ke controller. Ini mencegah SQL injection (Eloquent ORM sudah parametrized), XSS, dan data corrupt.

```
// Contoh validasi ketat di QuizSubmitRequest
public function rules(): array {
    return [
        'topic_id'           => 'required|exists:topics,id',
        'answers'            => 'required|array|min:1',
        'answers.*.question_id' => 'required|exists:questions,id',
        'answers.*.selected_answer' => 'required|in:A,B,C,D',
    ];
}
```

5.3 — Rate Limiting

Laravel menyediakan built-in rate limiting via throttle middleware. Konfigurasi di RouteServiceProvider atau langsung di route.

```
// routes/api.php — rate limit per endpoint sensitif
Route::middleware('throttle:5,1')->group(function () {
    // Max 5 request per menit untuk login
    Route::post('/login', [AuthController::class, 'login']);
});

// Default untuk semua API: 60 request per menit
Route::middleware('throttle:api')->group(function () {
    // semua route lainnya
});
```

5.4 — CORS Configuration

CORS dikonfigurasi di config/cors.php untuk mengizinkan request dari React (localhost:5173) dan Flutter. Di production, ganti dengan domain asli.

```
// config/cors.php
'allowed_origins' => [
    'http://localhost:5173',    // React dev
    'http://localhost:3000',    // React alt
    'https://structify.app',    // Production (nanti)
],
'allowed_methods' => ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
'allowed_headers' => ['Content-Type', 'Authorization', 'X-Requested-With'],
'supports_credentials' => true,
```

06 Deployment & DevOps

6.1 — Environment Strategy

Environment	Tujuan	APP_DEBUG	Database
local	Development harian	true	structify_db (local MySQL)
testing	Automated test	true	SQLite in-memory
staging	QA & demo	false	structify_staging_db
production	Live app	false	structify_production_db

6.2 — Deployment Checklist

Jalankan checklist ini setiap kali deploy ke staging/production:

- [] APP_DEBUG=false di .env production
- [] APP_KEY sudah di-generate (php artisan key:generate)
- [] Database credentials benar di .env
- [] php artisan migrate --force (jalankan migration)
- [] php artisan db:seed --class=TrackSeeder (jika fresh deploy)
- [] php artisan db:seed --class=TopicSeeder
- [] php artisan config:cache (cache konfigurasi)
- [] php artisan route:cache (cache routes)
- [] php artisan view:cache (cache views)
- [] php artisan storage:link (symlink storage)
- [] Pastikan CORS allowed_origins sudah update ke domain production
- [] Test endpoint /api/tracks via Postman (sanity check)
- [] Test endpoint /api/login via Postman

6.3 — Monitoring & Logging

Laravel menyimpan log di storage/logs/laravel.log. Gunakan tail -f untuk monitor log real-time saat development.

```
# Monitor log real-time
tail -f storage/logs/laravel.log

# Clear log
echo '' > storage/logs/laravel.log

# Custom logging di controller
use Illuminate\Support\Facades\Log;
Log::info('Quiz submitted', ['user_id' => $user->id, 'score' => $score]);
```

```
Log::error('Quiz error', ['error' => $e->getMessage()]);
```

```
# Level: emergency, alert, critical, error, warning, notice, info, debug
```